FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 4: Functions, dictionaries, tuples

Tim Van de Cruys



Wednesday 14 October 2026

Functions

- In mathematics: mapping of an input to an output
- In the context of programming: a sequence of statements that we can execute together
- Why? Modularity:
  - Create re-usable components: we only need to write a complex piece of code once, and we can then reuse it
  - Create larger, more complex programs from easily understandable building blocks
  - Functions encapsulate complexity: we don't need to understand how a function works to use it

Functions

- We've already used builtin functions (`print`, `len`, `type`)
- Functions that are already defined by default, and which can be called using a **function call**:

```
In: type(101)
Out: int
```

- A function has a **name**: `type`
- A function often takes an **argument** (in parentheses) given to the function as input
- A function often outputs a result: **return value**
- But: neither input nor output is necessary!
- We can also define our own functions

Defining functions

Let's look at some simple function definitions:

```
def print_refrain():
    print('Twinkle, twinkle, little star,')
    print('How I wonder what you are!')
```

```
def print_stanza():
    print('Up above the world so high,')
    print('Like a diamond in the sky.')
```

- These functions do not take any arguments, and they do not return anything either
- Note: executing a `def` statement does not run the code!
- The function's code is just stored in memory, along with its name

Calling functions

- Function must be called by its name in order to run it:

```
In:  print_refrain()
Out:
```

```
Twinkle, twinkle, little star,
How I wonder what you are!
```

```
In:  print_stanza()
Out:
```

```
Up above the world so high,
Like a diamond in the sky.
```

Re-using functions

- Our functions can now be reused, e.g. within another function:

```
def print_full_lyrics():
    print_refrain()
    print_stanza()
    print_refrain()
```

```
In:  print_full_lyrics()
Out:
```

```
Twinkle, twinkle, little star,
How I wonder what you are!
Up above the world so high,
Like a diamond in the sky.
Twinkle, twinkle, little star,
How I wonder what you are!
```

Function arguments

- Some functions take arguments
- Inside the function, the arguments are assigned to variables called **parameters**
- A *parameter* is a value expected by a function
- An *argument* is a particular value for a parameter when calling a function

Basic syntax:

```
def name(parameter1, parameter2, ...):
    # do something with the parameters
```

Function arguments

- We can provide data to functions using arguments
- Below, the argument `'David'` is assigned to the parameter `name` when the function is called
- Arguments have a specific order
- When calling a function, we need to use correct number of arguments

Example:

```
def display_greeting(name):
    print('Hello', name)

display_greeting('David')
```

Returning results

- Most functions compute a result based on an input
- You probably want to do something with that result
- In order to make the result available, you need a `return` statement
- `return result` returns the result back to the caller (such that it can be assigned to a variable)
- Variables within the function have local **scope**

Example:

```
def add_smiley(sentence):
    new_sentence = sentence + ' :)'
    return new_sentence
```

Returning results

- Note: if no `return` statement is provided, the function implicitly returns `None`
- Note: `return` can occur anywhere in the function, which immediately stops its execution

Returning results

- Note: if no `return` statement is provided, the function implicitly returns `None`
- Note: `return` can occur anywhere in the function, which immediately stops its execution

```
def say_hello():
    print('Hello')

a = say_hello()
print(a)
```

Returning results

- Note: if no `return` statement is provided, the function implicitly returns `None`
- Note: `return` can occur anywhere in the function, which immediately stops its execution

```
def say_hello():
    print('Hello')

a = say_hello()
print(a)
```

Result:

```
None
```

Returning results

- Note: if no `return` statement is provided, the function implicitly returns `None`
- Note: `return` can occur anywhere in the function, which immediately stops its execution

```
def say_hello():
    print('Hello')

a = say_hello()
print(a)
```

Result:

```
None
```

```
def even_odd(number):
    if number % 2 == 0:
        return 'even'
    else:
        return 'odd'

result = even_odd(5)
print(result)
```

Returning results

- Note: if no `return` statement is provided, the function implicitly returns `None`
- Note: `return` can occur anywhere in the function, which immediately stops its execution

```
def say_hello():
    print('Hello')

a = say_hello()
print(a)
```

Result:

```
None
```

```
def even_odd(number):
    if number % 2 == 0:
        return 'even'
    else:
        return 'odd'

result = even_odd(5)
print(result)
```

Result:

```
odd
```

Functions: summary

- Break programs down into functions to make them reusable
- Define a function using `def` with a name, parameters, and a block of code
- Defining a function does not run it
- Arguments in a call are matched to parameters in definition
- Functions may return a result to their caller using `return`

Dictionaries

- Lists are useful when data has a natural order, or we need to go through it sequentially: get items by their position
- What if we have lots of data, but we need to find it some other way?
- Example: finding words in a dictionary
  - Too many words to consider one-by-one
  - However, because they are sorted alphabetically, we can find words quickly

Dictionaries

- A dictionary stores (key, value) pairs, such that given a key, we can quickly find its value

Example, a phone book:

```
>>> coworkers = {'Michael': 123, 'Alice': 456, 'Kate': 789}
>>> coworkers['Alice']
456
>>> coworkers['Bob'] = 124
```

- Even for lots of data, dictionaries are very quick
- implemented using ‘hash tables’

Properties of dictionaries

A dictionary is a mapping of keys to values.

- Looking up a key is fast
- Keys must be immutable (e.g., numbers or strings)
- Values can be anything
- Keys are always unique: re-using a key will overwrite the previous value

Working with dictionaries

```
In: coworkers = {'Michael': 123, 'Alice': 456}
In: coworkers['Alice']
Out: 456
In: coworkers['Bob'] = 124
In: 'Alice' in coworkers
Out: True
In: len(coworkers)
Out: 3
In: del coworkers['Michael']
In: coworkers['Kate']
KeyError: 'Kate'
```

- `in` tests whether a **key** exists; looking up a key that does not exist gives a `KeyError`
- `.keys()`, `.values()` and `.items()` give the keys, the values, and the (key, value) pairs

Lists vs. dictionaries: practical example

Two ways of organizing a collection of books:

```
books = ['Tolstoy - War and Peace',
    'Tolstoy - Anna Karenina',
    'Dostoevsky - Crime and Punishment',
    'Dostoevsky - Brothers Karamazov']
```

Lists vs. dictionaries: practical example

Two ways of organizing a collection of books:

```
books = ['Tolstoy - War and Peace',
    'Tolstoy - Anna Karenina',
    'Dostoevsky - Crime and Punishment',
    'Dostoevsky - Brothers Karamazov']
```

```
books = {'Tolstoy': ['War and Peace', 'Anna Karenina'],
    'Dostoevsky': ['Crime and Punishment', 'Brothers Karamazov']}
print(books['Tolstoy'])
```

- Easier to select works by a particular author

Using a dictionary to count

You are given a string, and need to count how many times each letter appears. How to solve?

1. Create 26 variables (`a`, `b`, `c`, ...), one for each letter. Initialize each variable to zero; loop through the string, and increase the variables for the correct letter (using conditional expression)
2. List with 26 elements; convert each character to a number, and use the number to index into the list (not straightforward, but possible)
3. Create a **dictionary**; characters are the keys, and corresponding values act as counters

Using a dictionary to count

- Let's try that last option:

```
word = 'interdisciplinary'
character_counter = {}
for char in word:
    if char not in character_counter:
        character_counter[char] = 1
    else:
        character_counter[char] += 1
print(character_counter)
```

Counting words

```
sentence = """
As he crossed toward the pharmacy at the corner he involuntarily
turned his head because of a burst of light that had ricocheted from
his temple and saw with that quick smile with which we greet a
rainbow or a rose a blindingly white parallelogram of sky being
unloaded from the van a dresser with mirrors across which as across a
cinema screen  passed a flawlessly clear reflection of boughs sliding
and swaying not arboreally but with a human vacillation produced by
the nature of those who were carrying this sky these boughs this
gliding facade
"""
```

(Vladimir Nabokov, The Gift)

Counting words

```
word_list = sentence.lower().split()
word_counter = {}
for word in word_list:
    if word not in word_counter:
        word_counter[word] = 1
    else:
        word_counter[word] += 1
```

Looping over dictionaries

Looping over a dictionary returns only the keys:

```
data = {'a': 0, 'b': 1}
for key in data:
    print(key)
    print(data[key])
```

Looping over dictionaries

Looping over a dictionary returns only the keys:

```
data = {'a': 0, 'b': 1}
for key in data:
    print(key)
    print(data[key])
```

Often want both the key and the value:

```
for key, value in data.items():
    print(key)
    print(value)
```

How to print out words (and their frequencies) in alphabetical order?

How to print words with frequency of at least 4?

Tuples

- Lists are mutable, so you cannot use them as keys in dictionary!
- Tuples are similar to lists but are immutable

Create a tuple with parentheses:

```
couple = ('John', 'Mary')
years_married = {('John', 'Mary'): 3, ('Alice', 'Bob'): 5}
```

Tuples

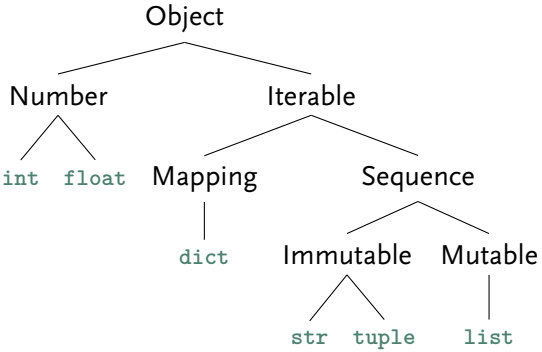
```
In: couple = ('John', 'Mary')
In: couple[0]
Out: 'John'
In: couple[0] = 'Jim'
TypeError: 'tuple' object does not support item assignment
In: name, partner = couple
In: partner
Out: 'Mary'
In: (3, 'the') < (5, 'a')
Out: True
In: (3, 'a') < (3, 'the')
Out: True
```

- Indexing works as for lists, but items cannot be changed
- **Unpacking**: assign the items to separate variables at once (this is what happens in `for key, value in data.items()`)
- Tuples are compared item by item: the first item decides, the second breaks ties

Sort words from longest to shortest

Get most common words in a text

Summary of data types



Practical session / reading

- Practical session 3
- The code-along notebook of this lecture is on Toledo: re-run the examples at home
- Read chapters 7 & 8 of Python for Everybody